

About Virus Breach

Virus Breach is a 2D arcade style game running on the JavaFX engine. The player acts as a virus who is infiltrating a server environment to steal sensitive data. 6 packages of data are required to exit, and along the way source codes may be collected for extra data. In the server, an antivirus will be chasing the virus and will immediately quarantine the virus on contact. The virus should also avoid firewalls, which will corrupt and prevent data from being exfiltrated. The secured server room at the top left of the map is unlockable with a decryption key, obtained in the storage room at the bottom right.

Design Improvements

We were quite faithful to our original design. We added some new features to our game in the later phases but never had to change our game's design significantly. Some of the things we changed are as follows:

1. The firewalls around the map were made to be spreadable. The original design included only static firewalls spawned at the start of the game that were easy to avoid. This change was made to ramp up the game's difficulty as time goes on, to pressure the player into taking action more quickly.
2. As the firewalls spread, however, we noticed that the spread became too fast and can sometimes make the game too difficult. The item "freeze token" is added to temporarily pause the spread. This change creates a set strategy for the game, where the player should optimally try to obtain the token as quickly as possible.
3. In the original design, the antivirus would move towards the player at every tick. However, since the player moves at the same speed, the antivirus would quickly catch up and never get further away. We deemed that this mechanic punished the player too harshly and did not give the player the opportunity to play around on the map. We decided to make the antivirus move every other tick instead, essentially making it go twice as slow.
4. There were not many ways to differentiate the 5 room types originally envisioned in our design that allowed us to make each of them special. So, we decided to scrap 3 of them and just keep the Server and Storage rooms in the game. We added the locked door and key system to make the rooms distinct from the main area, and added the freeze token within to make exploring the rooms more rewarding.
5. Class structures are largely the same. In the original design, the entity hierarchy is split into rewards (data/source code) and enemies (firewall/antivirus). However, it

was more logical to have all items (data/source code/firewall/key/token) extend Collectable instead, since they all share the same collision and handling logic.

6. Our original system was designed around an immediate-mode rendering library (e.g. LWJGL) where we would handle all of the draw calls. However, while implementing the game we realized we do not need to have such low-level control to meet our goals. So, instead of reinventing the wheel again we decided to go with a retained-mode graphics library with more features.

Lessons learned

1. We found out later into the implementation phase that the game engine library that we used, FXGL, was incompatible with the project requirements. Many aspects of the game's logic were built-in to the library, and it was not as object-oriented as we expected. The engine was tightly coupled with its UI and would have made it difficult to implement JUnit tests in phase 3. In the middle of phase 2, we performed an expensive refactor to convert the game to a different engine, JavaFX. This costly transition could have been avoided by doing deeper research and evaluation of which library we were going to use in the design phase of our project. We should have made prototypes to get a feel for each library before making such an important choice in our tech stack.
2. Throughout the development, especially after the implementation phase, there were a few overlapped responsibilities and unclear expectations set amongst us. We learned that it is important to assign tasks such that everyone can take ownership of an aspect of the project. Then, everyone can be confident in what their responsibilities are and plan out their time accordingly. We also learned that frequent and thorough communication is an important aspect of any project.

Video Demo

<https://youtu.be/TR7ZeQVZClo>

